

UNIVERSIDAD DE LAS FUERZAS ARMADAS-ESPE

SEDE SANTO DOMINGO DE LOS TSÁCHILAS



**SEDE
SANTO DOMINGO**

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN - DCCO-SS

CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN



PERIODO	:	202650 abril – agosto 2026
ASIGNATURA	:	Fundamentos de Sistemas Web
TEMA	:	Taller: Construcción de interacciones web
ESTUDIANTES	:	Liscano Carbo Jhon Anderson
NIVEL-PARALELO – NRC	:	Cuarto - 29535
DOCENTE	:	Ing. Geovanny Brito
FECHA DE ENTREGA	:	17 de julio de 2026

SANTO DOMINGO – ECUADOR

1. Índice

1. Índice	2
2. Índice de figuras	3
3. Enlace del repositorio GitHub	4
4. Descripción de la actividad	4
5. Objetivo general	4
5.1. Objetivos específicos	4
6. Marco conceptual.	4
7. Desarrollo paso a paso.	5
7.1. Ejercicio 1 — Agregar la cuarta nota	5
7.2. Ejercicio 2 — Notas con decimales y promedio con dos decimales	6
7.3. Ejercicio 3 — Ajustar las carreras del formulario	7
7.4. Ejercicio 4 — Beca académica	7
7.5. Ejercicio 5 — Colores según el tipo de beca	7
7.6. Ejercicio 6 — Promedio especial sin la nota más baja	9
7.7. Ejercicio 7 — Nota más alta y nota más baja	10
7.8. Ejercicio 8 — Contar notas aprobadas, en recuperación y reprobadas	11
7.9. Ejercicio 9 — Recomendación académica	11
7.10. Ejercicio 10 — Clasificar el rendimiento académico	12
7.11. Ejercicio 11 — Historial de estudiantes evaluados	12
7.12. Ejercicio 12 — Validar nombres repetidos	12
7.13. Ejercicio 13 — Botón para listar estudiantes	13
7.14. Ejercicio 14 — Contador de estudiantes evaluados	13
7.15. Ejercicios 15 y 16 — Estadísticas del curso	13
7.16. Ejercicio 17 — Buscar estudiante por nombre	14
7.17. Ejercicio 18 — Ranking de estudiantes	14
7.18. Ejercicio 19 — Estudiante con mayor y menor promedio	15
7.19. Ejercicio 20 — Objeto estudiante completo	15
8. Pruebas realizadas	16
9. Funciones implementadas	18
10. Análisis de resultados	19
11. Conclusiones	19
12. Recomendaciones	20

2. Índice de figuras

Figura 1. Formulario académico con la cuarta nota agregada.	5
Figura 2. Mensaje de beca calculada según carrera y promedio con color correspondiente.	7
Figura 3. Resultado de evaluación mostrando estado, rendimiento, nota máxima y mínima.	8
Figura 4. Estadísticas generales del curso con mejor y menor promedio.	10
Figura 5. Búsqueda de estudiante por nombre con fila resaltada.	10
Figura 6. Tabla del historial de estudiantes evaluados con ranking.	11
Figura 7. Salida del objeto estudiante completo en formato JSON.	11

3. Enlace del repositorio GitHub

[jaliscano-oss/Taller Interacciones Web](https://github.com/jaliscano-oss/Taller_Interacciones_Web)

4. Descripción de la actividad

Este taller consistió en ampliar un evaluador académico base desarrollado en clase. Partiendo de un formulario HTML con tres notas y una lógica JavaScript inicial, se construyeron 20 nuevas funcionalidades de forma progresiva. El objetivo fue aplicar en la práctica los conceptos de funciones, eventos, arreglos, objetos, estructuras condicionales, ciclos y manipulación del DOM, conectando cada cálculo a la interfaz visual mediante componentes de Bootstrap 5.

5. Objetivo general

Desarrollar una aplicación web interactiva que amplíe el evaluador académico base, incorporando funcionalidades de cálculo, validación, historial, ranking y estadísticas, documentando el proceso de forma técnica y reflexiva.

5.1. Objetivos específicos

- Agregar una cuarta nota al formulario y ajustar toda la lógica de cálculo.
- Implementar soporte para notas con decimales y mostrar el promedio con dos cifras decimales.
- Ajustar el selector de carrera a las opciones: Tecnologías de la Información, Software y Sistemas.
- Crear un mensaje de beca con color diferenciado por porcentaje usando clases Bootstrap.
- Calcular un promedio especial excluyendo la nota más baja.
- Identificar la nota máxima y mínima de cada estudiante.
- Contar cuántas notas individuales son aprobadas, de recuperación o reprobadas.
- Generar una recomendación académica y clasificar el rendimiento según el promedio.
- Construir un historial global con validación de nombres repetidos.
- Mostrar un ranking ordenado por promedio y estadísticas generales del curso.
- Implementar búsqueda de estudiantes por nombre con resaltado visual.
- Presentar el objeto completo del estudiante en formato JSON.

6. Marco conceptual.

Manipulación del DOM

El Document Object Model (DOM) es la representación en memoria de un documento HTML. JavaScript permite acceder a sus elementos mediante `document.getElementById()`, modificar su contenido con `.textContent` y controlar su visibilidad con `.classList.add()` / `.classList.remove()`.

Funciones en JavaScript

Una función es un bloque de código reutilizable que recibe parámetros y puede devolver un valor. En este proyecto se usaron funciones declaradas con `function`, siguiendo el principio de responsabilidad única: cada función realiza una sola tarea específica.

Arreglos y objetos

Un arreglo (Array) almacena múltiples valores en una sola variable y permite recorrerlos con bucles for. Un objeto agrupa propiedades relacionadas bajo una misma entidad. En este proyecto, cada estudiante se representa como un objeto y todos se acumulan en un arreglo global llamado estudiantes.

Eventos

Los eventos permiten ejecutar código en respuesta a acciones del usuario. Se usó `addEventListener("click", función)` para asociar los botones Evaluar, Limpiar, Ver historial y Buscar a sus respectivas funciones.

Bootstrap 5

Bootstrap es un framework CSS que permite construir interfaces responsivas mediante clases predefinidas como `alert`, `table`, `d-none`, `btn` y el sistema de grillas `col-md-*`. Las clases `alert-success`, `alert-primary`, `alert-warning` y `alert-secondary` se usaron para diferenciar visualmente los mensajes de beca.

7. Desarrollo paso a paso.

7.1. Ejercicio 1 — Agregar la cuarta nota

Se agregó el campo `nota4` en el HTML dentro de la grilla Bootstrap con `col-md-4`. En `app.js` se incluyó la lectura del nuevo campo, su validación en la condición de campos vacíos y su incorporación al arreglo `notas[]`.

```
<div class="col-md-4">

  <label for="nota4" class="form-label">Nota 4</label>

  <input id="nota4" type="number" class="form-control" min="0"
max="20" step="0.01">

</div>
```

Laboratorio JS - Fundamentos lógicos

http://127.0.0.1:3000/index.html?serverWindowId=654c539d-61c4-4640-8aa6-574bd90c

Evaluador académico

Ingrese los datos y calcule el estado del estudiante.

Nombre

Ejemplo: Ana Torres

Carrera

Seleccione...

Nota 1

Nota 2

Nota 3

Nota 4

Evaluar Limpiar Ver historial

Figura 1. Formulario académico con la cuarta nota agregada.

7.2. Ejercicio 2 — Notas con decimales y promedio con dos decimales

Los campos de notas tienen el atributo `step="0.01"` que habilita la entrada de decimales. En JavaScript, el método `toFixed(2)` garantiza que el promedio siempre se muestre con exactamente dos cifras decimales.

```
`tiene un promedio de ${estudiante.promedio.toFixed(2)}`
```

7.3. Ejercicio 3 — Ajustar las carreras del formulario

Se modificó el selector `<select id="carrera">` para reemplazar la opción Redes por Sistemas, que es la carrera correcta según el enunciado del taller.

```
<option value="TI">Tecnologías de la Información</option>
<option value="Software">Software</option>
<option value="Sistemas">Sistemas</option>
```

7.4. Ejercicio 4 — Beca académica

Se creó la función `calcularBeca(carrera, promedio)` que evalúa la carrera y el promedio del estudiante usando operadores relacionales estrictos (`>`) para determinar el porcentaje de beca que corresponde.

```
function calcularBeca(carrera, promedio) {
  if (carrera === "TI" && promedio > 18) {
    return 100;
  } else if (carrera === "Software" && promedio > 17) {
    return 80;
  } else if (carrera === "Sistemas" && promedio > 16) {
    return 60;
  } else {
    return null;
  }
}
```

7.5. Ejercicio 5 — Colores según el tipo de beca

La función `mostrarBeca(beca)` asigna una clase Bootstrap diferente al `divBeca` según el porcentaje obtenido: verde para 100%, azul para 80%, amarillo para 60% y gris si no hay beca. El div siempre es visible para informar al usuario del resultado.

```
function mostrarBeca(beca) {
  const divBeca = document.getElementById("divBeca");
  if (beca === 100) {
    divBeca.className = "alert alert-success mt-4";
    divBeca.textContent = "¡Felicitaciones! Este estudiante
obtiene una beca del 100%.";
  } else if (beca === 80) {
    divBeca.className = "alert alert-primary mt-4";
    divBeca.textContent = "¡Felicitaciones! Este estudiante
```

```
obtiene una beca del 80%.";
    } else if (beca === 60) {
        divBeca.className = "alert alert-warning mt-4";
        divBeca.textContent = "¡Felicitaciones! Este estudiante
obtiene una beca del 60%.";
    } else {
        divBeca.className = "alert alert-secondary mt-4";
        divBeca.textContent = "Este estudiante no califica para
beca en este período.";
    }
    divBeca.classList.remove("d-none");
}
```

Evaluador académico

Ingrese los datos y calcule el estado del estudiante.

Nombre

Jhon Liscano

Carrera

Tecnologías de la Información

Nota 1

18

Nota 2

18

Nota 3

18

Nota 4

18

Evaluar

Limpiar

Ver historial

¡Felicitaciones! Este estudiante obtiene una beca del 100%.

El estudiante Jhon Liscano, de la carrera TI, tiene un promedio de 18.00 y su estado es: Aprobado. Rendimiento: Alto. Nota más alta: 18 | Nota más baja: 18. Promedio sin mínima: 18.00. Aprobadas: 4 | Recuperación: 0 | Reprobadas: 0. Recomendación: Mantener el desempeño y apoyar a compañeros.

```
[
  {
    "nombre": "Jhon Liscano",
    "carrera": "TI",
    "notas": [
      18,
      18,
      18,
      18
    ],
    "promedio": 18,
    "promedioSinMinima": 18,
    "notaMaxima": 18,
    "notaMinima": 18,
    "estado": "Aprobado",
    "rendimiento": "Alto",
    "recomendacion": "Mantener el desempeño y apoyar a compañeros.",
    "notasAprobadas": 4,
    "notasRecuperacion": 0,
    "notasReprobadas": 0,
    "beca": 100
  }
]
```

Activar Windows

Ve a Configuración para activar Windows.

Estudiantes evaluados: 1

Figura 2. Mensaje de beca calculada según carrera y promedio con color correspondiente.

7.6. Ejercicio 6 — Promedio especial sin la nota más baja

La función `calcularPromedioSinMinima(notas)` primero obtiene la nota mínima usando `obtenerExtremos`. Luego recorre el arreglo con una variable booleana `excluido` para saltar solo la primera aparición de ese valor mínimo, evitando excluir notas iguales que no corresponde eliminar.

```
function calcularPromedioSinMinima(notas) {
  const { min } = obtenerExtremos(notas);
```

```
    let suma = 0;
    let excluido = false;
    for (const nota of notas) {
        if (nota === min && !excluido) {
            excluido = true;
            continue;
        }
        suma += nota;
    }
    return suma / (notas.length - 1);
}
```

7.7. Ejercicio 7 — Nota más alta y nota más baja

La función `obtenerExtremos(notas)` recorre el arreglo una sola vez comparando cada nota contra los valores actuales de `max` y `min`. Devuelve un objeto con ambos valores, que luego se desestructura con `const { max, min } = obtenerExtremos(notas)`.

```
function obtenerExtremos(notas) {
    let max = notas[0];
    let min = notas[0];
    for (const nota of notas) {
        if (nota > max) {
            max = nota;
        }
        if (nota < min) {
            min = nota;
        }
    }
    return { max, min };
}
```

El estudiante Jhon Liscano, de la carrera TI, tiene un promedio de 18.00 y su estado es: Aprobado. Rendimiento: Alto. Nota más alta: 18 | Nota más baja: 18. Promedio sin mínima: 18.00. Aprobadas: 4 | Recuperación: 0 | Reprobadas: 0. Recomendación: Mantener el desempeño y apoyar a compañeros.

```
[
  {
    "nombre": "Jhon Liscano",
    "carrera": "TI",
    "notas": [
      18,
      18,
      18,
      18
    ],
    "promedio": 18,
    "promedioSinMinima": 18,
    "notaMaxima": 18,
    "notaMinima": 18,
    "estado": "Aprobado",
    "rendimiento": "Alto",
    "recomendacion": "Mantener el desempeño y apoyar a compañeros"
```

Figura 3. Resultado de evaluación mostrando estado, rendimiento, nota máxima y mínima.

7.8. Ejercicio 8 — Contar notas aprobadas, en recuperación y reprobadas

La función `clasificarNotas(notas)` recorre el arreglo y acumula tres contadores usando los mismos rangos que `clasificarEstado`. Devuelve un objeto con los tres valores que luego se muestran en el mensaje de resultado.

```
function clasificarNotas(notas) {
  let aprobadas = 0;
  let recuperacion = 0;
  let reprobadas = 0;
  for (const nota of notas) {
    if (nota >= 14) {
      aprobadas++;
    } else if (nota >= 10) {
      recuperacion++;
    } else {
      reprobadas++;
    }
  }
  return { aprobadas, recuperacion, reprobadas };
}
```

7.9. Ejercicio 9 — Recomendación académica

La función `obtenerRecomendacion(promedio)` usa una cadena `if / else if / else` para devolver un mensaje orientador según cuatro rangos de promedio definidos en el enunciado.

```
function obtenerRecomendacion(promedio) {
  if (promedio >= 18) {
    return "Mantener el desempeño y apoyar a compañeros.";
  } else if (promedio >= 14) {
    return "Reforzar temas específicos.";
  } else if (promedio >= 10) {
    return "Asistir a tutorías y practicar ejercicios.";
  } else {
    return "Repetir contenidos base y solicitar acompañamiento.";
  }
}
```

7.10. Ejercicio 10 — Clasificar el rendimiento académico

La función `clasificarRendimiento(promedio)` devuelve una de cuatro categorías: Alto (≥ 18), Medio (≥ 14), Básico (≥ 10) o Bajo (< 10). Este valor se incluye en el objeto estudiante y se muestra tanto en el mensaje de resultado como en la tabla del historial.

```
function clasificarRendimiento(promedio) {
  if (promedio >= 18) {
    return "Alto";
  } else if (promedio >= 14) {
    return "Medio";
  } else if (promedio >= 10) {
    return "Básico";
  } else {
    return "Bajo";
  }
}
```

7.11. Ejercicio 11 — Historial de estudiantes evaluados

Se declaró el arreglo global `const estudiantes = []` al inicio del archivo. Cada vez que se evalúa un estudiante correctamente, se usa `estudiantes.push(estudiante)` para agregar el objeto al historial. El arreglo persiste en memoria durante toda la sesión del navegador.

7.12. Ejercicio 12 — Validar nombres repetidos

La función `existeNombreRepetido(nombre)` recorre el arreglo `estudiantes` comparando nombres en minúsculas para evitar que mayúsculas o minúsculas generen duplicados. Si encuentra coincidencia, devuelve `true` y `evaluarEstudiante` interrumpe el proceso mostrando un mensaje de advertencia.

```
function existeNombreRepetido(nombre) {  
  for (const est of estudiantes) {  
    if (est.nombre.toLowerCase() === nombre.toLowerCase()) {  
      return true;  
    }  
  }  
  return false;  
}
```

7.13. Ejercicio 13 — Botón para listar estudiantes

Se agregó el botón "Ver historial" en el HTML con el atributo `onclick="mostrarTablaHistorial()"`. Al pulsarlo, la función genera las filas de la tabla dinámicamente con `document.createElement("tr")` y las inserta en el `<tbody id="tablaHistorialBody">`.

7.14. Ejercicio 14 — Contador de estudiantes evaluados

La función `actualizarContadorHistorial()` se llama cada vez que se agrega un estudiante al historial. Actualiza el texto del párrafo `<p id="contadorHistorial">` con el total actual de `estudiantes.length`.

```
function actualizarContadorHistorial() {  
  const contador = document.getElementById("contadorHistorial");  
  contador.textContent = `Estudiantes evaluados:  
${estudiantes.length}`;  
}
```

7.15. Ejercicios 15 y 16 — Estadísticas del curso

La función `mostrarEstadisticasGenerales(ranking)` recorre el arreglo `estudiantes` y acumula contadores por estado. Luego calcula el promedio general dividiendo la suma de promedios entre la cantidad total de estudiantes.

```
const promedioGeneral = sumaPromedios / estudiantes.length;
```

Los resultados se muestran en los elementos `<strong>` de la sección de estadísticas.

Estadísticas generales
Total evaluados: 4
Aprobados: 4
Recuperación: 0
Reprobados: 0
Promedio general del curso: 16.97
Mejor promedio: Jhon Liscano (18.00)
Menor promedio: Sanchez (15.86)

Figura 4. Estadísticas generales del curso con mejor y menor promedio.

7.16. Ejercicio 17 — Buscar estudiante por nombre

La función `buscarEstudiante()` primero abre la tabla con `mostrarTablaHistorial()` para garantizar que las filas existan en el DOM. Luego recorre cada fila y compara el texto de la segunda celda (nombre) con el término buscado, aplicando o quitando la clase `table-warning` para resaltar coincidencias.

Estudiantes evaluados: 4

Estadísticas generales
Total evaluados: 4
Aprobados: 4
Recuperación: 0
Reprobados: 0
Promedio general del curso: 16.97
Mejor promedio: Jhon Liscano (18.00)
Menor promedio: Sanchez (15.86)

Ranking de estudiantes evaluados

#	Nombre	Carrera	Notas	Promedio	Prom. sin mínima	Estado	Rendimiento	Recomendación	Beca
1	Jhon Liscano	TI	18, 18, 18, 18	18.00	18.00	Aprobado	Alto	Mantener el desempeño y apoyar a compañeros.	100%
2	Anderson	TI	14, 18, 18, 18	17.00	18.00	Aprobado	Medio	Reforzar temas específicos.	—
3	Tevez	TI	14, 18, 18, 18	17.00	18.00	Aprobado	Medio	Reforzar temas específicos.	—
4	Sanchez	TI	14, 13.45, 18, 18	15.86	16.67	Aprobado	Medio	Reforzar temas específicos.	—

Figura 5. Búsqueda de estudiante por nombre con fila resaltada.

7.17. Ejercicio 18 — Ranking de estudiantes

Dentro de `mostrarTablaHistorial()` se crea una copia del arreglo original con el operador `spread` `[...estudiantes]` y se ordena con `.sort()` de mayor a menor promedio. Se usa una copia para no alterar el orden de inserción del arreglo base.

```
const ranking = [...estudiantes].sort(function(a, b) {
  return b.promedio - a.promedio;
```

```
});
```

Ranking de estudiantes evaluados

#	Nombre	Carrera	Notas	Promedio	Prom. sin mínima	Estado	Rendimiento	Recomendación	Beca
1	Jhon Liscano	TI	18, 18, 18, 18	18.00	18.00	Aprobado	Alto	Mantener el desempeño y apoyar a compañeros.	100%
2	Anderson	TI	14, 18, 18, 18	17.00	18.00	Aprobado	Medio	Reforzar temas específicos.	—
3	Tevez	TI	14, 18, 18, 18	17.00	18.00	Aprobado	Medio	Reforzar temas específicos.	—
4	Sanchez	TI	14, 13.45, 18, 18	15.86	16.67	Aprobado	Medio	Reforzar temas específicos.	—

Figura 6. Tabla del historial de estudiantes evaluados con ranking.

7.18. Ejercicio 19 — Estudiante con mayor y menor promedio

Como el arreglo ranking ya está ordenado de mayor a menor, el estudiante con mejor promedio es ranking[0] y el de menor promedio es ranking[ranking.length - 1]. Ambos se muestran en la sección de estadísticas generales.

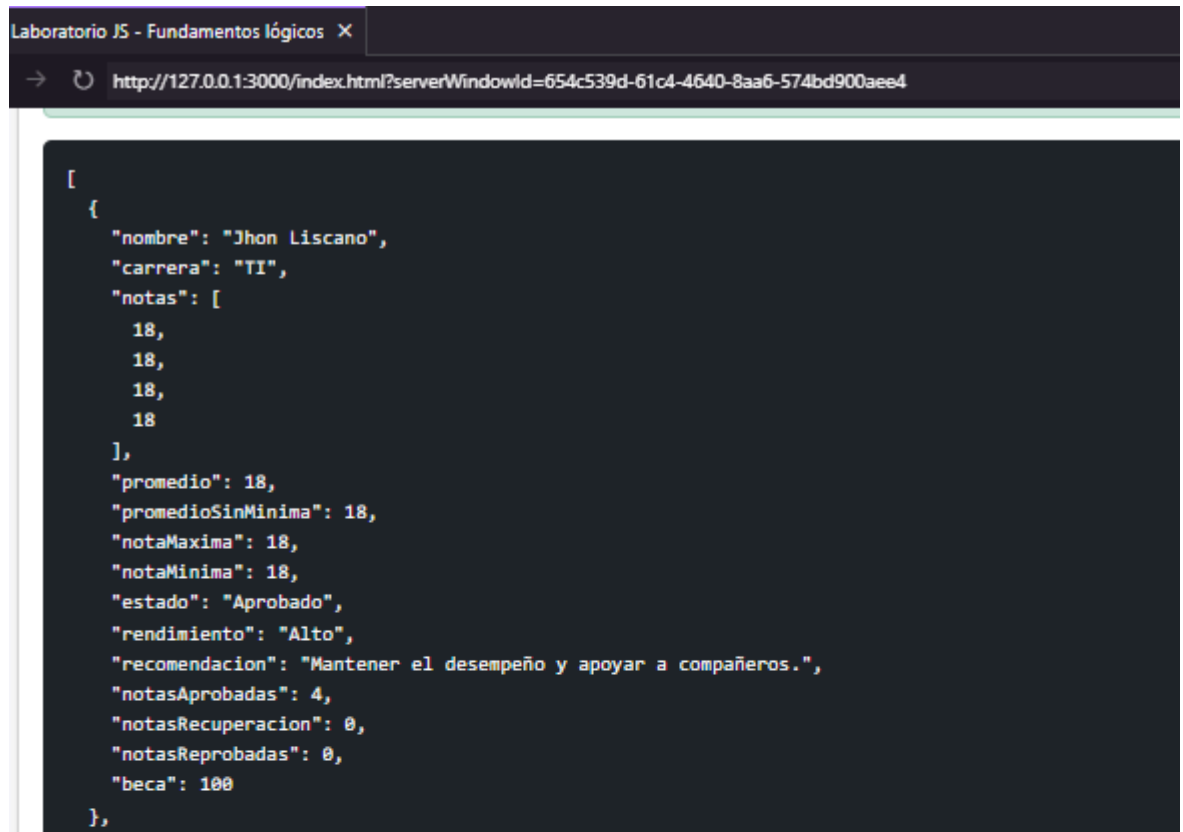
```
const mejorEstudiante = ranking[0];  
const peorEstudiante = ranking[ranking.length - 1];
```

7.19. Ejercicio 20 — Objeto estudiante completo

El objeto estudiante construido en evaluarEstudiante() incluye todos los campos requeridos por el enunciado. Se serializa con JSON.stringify(estudiantes, null, 2) y se muestra en el elemento <pre id="salidaJSON">.

```
const estudiante = {  
  nombre: nombre,  
  carrera: carrera,  
  notas: notas,  
  promedio: promedio,  
  promedioSinMinima: promedioSinMinima,  
  notaMaxima: max,  
  notaMinima: min,  
  estado: estado,  
  rendimiento: rendimiento,  
  recomendacion: recomendacion,  
  notasAprobadas: aprobadas,  
  notasRecuperacion: recuperacion,  
  notasReprobadas: reprobadas,  
  beca: beca
```

```
};
```



The screenshot shows a web browser window with the title "Laboratorio JS - Fundamentos lógicos". The address bar displays the URL "http://127.0.0.1:3000/index.html?serverWindowId=654c539d-61c4-4640-8aa6-574bd900aee4". The main content area shows a JSON object representing a student's data.

```
[
  {
    "nombre": "Jhon Liscano",
    "carrera": "TI",
    "notas": [
      18,
      18,
      18,
      18
    ],
    "promedio": 18,
    "promedioSinMinima": 18,
    "notaMaxima": 18,
    "notaMinima": 18,
    "estado": "Aprobado",
    "rendimiento": "Alto",
    "recomendacion": "Mantener el desempeño y apoyar a compañeros.",
    "notasAprobadas": 4,
    "notasRecuperacion": 0,
    "notasReprobadas": 0,
    "beca": 100
  },
]
```

Figura 7. Salida del objeto estudiante completo en formato JSON.

8. Pruebas realizadas

#	Caso de prueba	Entrada	Resultado esperado	Resultado obtenido
1	Campos vacíos	Sin ingresar datos	Mensaje de error rojo	Correcto
2	Nota fuera de rango	nota = 25	Mensaje de error rojo	Correcto
3	Estudiante aprobado	Promedio = 16.50	Estado: Aprobado, verde	Correcto
4	Estudiante recuperación	Promedio = 11.00	Estado: Recuperación, amarillo	Correcto
5	Estudiante reprobado	Promedio = 7.50	Estado: Reprobado, rojo	Correcto

#	Caso de prueba	Entrada	Resultado esperado	Resultado obtenido
6	Rendimiento Alto	Promedio = 19.00	Rendimiento: Alto	Correcto
7	Rendimiento Medio	Promedio = 15.00	Rendimiento: Medio	Correcto
8	Rendimiento Básico	Promedio = 11.00	Rendimiento: Básico	Correcto
9	Rendimiento Bajo	Promedio = 8.00	Rendimiento: Bajo	Correcto
10	Beca TI 100%	Carrera TI, promedio = 19	Beca 100%, fondo verde	Correcto
11	Beca Software 80%	Carrera Software, promedio = 18	Beca 80%, fondo azul	Correcto
12	Beca Sistemas 60%	Carrera Sistemas, promedio = 17	Beca 60%, fondo amarillo	Correcto
13	Sin beca	Promedio = 12	Mensaje gris sin beca	Correcto
14	Nombre repetido	Mismo nombre dos veces	Mensaje de advertencia	Correcto
15	Promedio sin mínima	Notas: 10, 15, 18, 20	Promedio sin 10: 17.67	Correcto
16	Ranking	Tres estudiantes distintos	Ordenados de mayor a menor	Correcto
17	Búsqueda existente	Nombre parcial en historial	Fila resaltada amarilla	Correcto
18	Búsqueda inexistente	Nombre no registrado	Mensaje "No se encontró"	Correcto
19	Estadísticas generales	Tres estudiantes evaluados	Totales y promedio correctos	Correcto

#	Caso de prueba	Entrada	Resultado esperado	Resultado obtenido
20	JSON completo	Evaluar un estudiante	Objeto con todos los campos	Correcto

9. Funciones implementadas

Función	Ejercicio	Descripción
evaluarEstudiante()	Base	Orquesta lectura, validación, cálculo y visualización
existeNotaInvalida(notas)	Base	Valida que cada nota esté entre 0 y 20
calcularPromedio(notas)	Base	Suma y divide entre la cantidad de notas
clasificarEstado(promedio)	Base	Devuelve Aprobado, Recuperación o Reprobado
obtenerColorEstado(estado)	Base	Devuelve la clase Bootstrap del color de estado
calcularPromedioSinMinima(notas)	6	Excluye la nota más baja y recalcula el promedio
obtenerExtremos(notas)	7	Devuelve nota máxima y mínima en un solo recorrido
clasificarNotas(notas)	8	Cuenta notas por categoría
obtenerRecomendacion(promedio)	9	Devuelve texto orientador según rango de promedio
clasificarRendimiento(promedio)	10	Devuelve Alto, Medio, Básico o Bajo
calcularBeca(carrera, promedio)	4	Calcula porcentaje de beca o null
mostrarBeca(beca)	5	Muestra beca con color Bootstrap según porcentaje
existeNombreRepetido(nombre)	12	Valida duplicados en el historial

Función	Ejercicio	Descripción
actualizarContadorHistorial()	14	Actualiza el total de evaluados en pantalla
mostrarTablaHistorial()	13, 18	Genera la tabla ordenada por ranking
mostrarEstadisticasGenerales(ranking)	15, 16, 19	Calcula totales, promedio general y extremos
buscarEstudiante()	17	Resalta filas que coincidan con el término buscado
construirMensaje(estudiante)	20	Arma el texto completo del resultado
mostrarJSON()	20	Serializa y muestra el arreglo de estudiantes
limpiarFormulario()	Base	Vacía todos los campos del formulario

10. Análisis de resultados

El evaluador académico cumple satisfactoriamente con los 20 ejercicios planteados. La organización del código en funciones con responsabilidad única facilita la lectura y el mantenimiento. Cada función realiza una tarea específica y devuelve un valor claro, lo que permite combinarlas en evaluarEstudiante() de forma ordenada.

El arreglo global estudiantes[] permite persistir los datos durante la sesión sin necesidad de servidor ni base de datos. La validación de nombres repetidos evita datos duplicados. El ranking se genera sobre una copia del arreglo con el operador spread, preservando el orden original de inserción.

La función calcularPromedioSinMinima maneja correctamente el caso de notas iguales al usar la variable booleana excluido, evitando que se salteen múltiples notas con el mismo valor mínimo.

El sistema de colores de beca usando clases Bootstrap aporta retroalimentación visual inmediata y consistente con el resto de la interfaz.

11. Conclusiones

- Separar la lógica en funciones con responsabilidad única facilita la escritura, prueba y corrección del código.
- Los objetos JavaScript permiten agrupar todos los datos de un estudiante en una sola entidad manejable, facilitando su almacenamiento, visualización en JSON y uso en tablas.

- Bootstrap 5 permite construir interfaces responsivas y con retroalimentación visual sin necesidad de CSS personalizado.
- La manipulación dinámica del DOM con `createElement` e `innerHTML` permite construir interfaces que responden a datos en tiempo real.
- Validar antes de procesar es una práctica esencial: las validaciones de campos vacíos, rango de notas y nombres repetidos garantizan la integridad del historial.
- El operador spread `[...arreglo]` es una herramienta útil para operar sobre copias sin modificar el arreglo original.

12. Recomendaciones

- Persistir el historial con `localStorage` para que los datos sobrevivan al recargar la página.
- Agregar un botón para eliminar estudiantes individuales del historial.
- Implementar exportación del historial a CSV para facilitar reportes académicos.
- Separar estilos personalizados en un archivo `styles.css` independiente.
- Para proyectos más grandes, considerar módulos ES6 para organizar el código en archivos separados por responsabilidad.

13. Criterio personal

Qué aprendí: cómo conectar la lógica de JavaScript directamente con la interfaz visual, haciendo que cada cálculo tenga una respuesta visible para el usuario.

Qué parte resultó más compleja: construir el historial con ranking y estadísticas, porque requería entender cómo los arreglos de objetos se ordenan y cómo obtener el primer y último elemento del resultado.

Qué funcionalidad considero más útil: las estadísticas generales del curso, porque transforman el evaluador de una herramienta individual a una herramienta de gestión grupal.

Cómo este taller me ayudó: comprender que una aplicación web no es solo diseño visual, sino lógica que responde a datos reales del usuario en tiempo real.

14. Bibliografía

MDN Web Docs. (2024). JavaScript Guide. Mozilla.
<https://developer.mozilla.org/es/docs/Web/JavaScript/Guide>

MDN Web Docs. (2024). Document Object Model (DOM). Mozilla.
https://developer.mozilla.org/es/docs/Web/API/Document_Object_Model

MDN Web Docs. (2024). `Array.prototype.sort()`. Mozilla.
https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/sort

Bootstrap. (2024). Bootstrap 5.3 Documentation. <https://getbootstrap.com/docs/5.3>

Flanagan, D. (2020). JavaScript: The Definitive Guide (7ma ed.). O'Reilly Media.

Haverbeke, M. (2018). Eloquent JavaScript (3ra ed.). No Starch Press.
<https://eloquentjavascript.net>